4_10xHuman_8CLC_comparison_geneIntersection

August 31, 2026

[1]:
```
library(rtracklayer)
library(ggplot2)
library(Seurat)

getwd()
dir.create("figures_10xHuman_8CLC")
dir.create("data")


colorTools = c("SoloTE"="#A4DD9B", #6fc69d",
               "Stellarscope"="#4f5d93",
               "STARsolo"="#f0df93")

dataset_id <- "10xHuman_8CLC"
```

```
Loading required package: GenomicRanges

Loading required package: stats4

Loading required package: BiocGenerics


Attaching package: ‘BiocGenerics’


The following objects are masked from ‘package:stats’:

    IQR, mad, sd, var, xtabs


The following objects are masked from ‘package:base’:

    anyDuplicated, aperm, append, as.data.frame, basename, cbind,
    colnames, dirname, do.call, duplicated, eval, evalq, Filter, Find,
    get, grep, grepl, intersect, is.unsorted, lapply, Map, mapply,
    match, mget, order, paste, pmax, pmax.int, pmin, pmin.int,
    Position, rank, rbind, Reduce, rownames, sapply, setdiff, sort,
    table, tapply, union, unique, unsplit, which.max, which.min
```

```
Loading required package: S4Vectors

Attaching package: ‘S4Vectors’

The following object is masked from ‘package:utils’:

    findMatches

The following objects are masked from ‘package:base’:

    expand.grid, I, unname

Loading required package: IRanges
Loading required package: GenomeInfoDb
Loading required package: SeuratObject
Loading required package: sp

Attaching package: ‘sp’

The following object is masked from ‘package:IRanges’:

    %over%


Attaching package: ‘SeuratObject’

The following object is masked from ‘package:GenomicRanges’:

    intersect

The following object is masked from ‘package:GenomeInfoDb’:

    intersect
```

```
The following object is masked from ‘package:IRanges’:

    intersect


The following object is masked from ‘package:S4Vectors’:

    intersect


The following object is masked from ‘package:BiocGenerics’:

    intersect


The following objects are masked from ‘package:base’:

    intersect, t
```

’/mnt/TEdata/TEbenchmarking/jupyterNotebooks/realDatasets’

```
Warning message in dir.create("figures_10xHuman_8CLC"):
"'figures_10xHuman_8CLC' already exists"
Warning message in dir.create("data"):
"'data' already exists"
```

[2]:
```
# theme_paper <- function(base_size = 17, base_family = "") {
#   theme_minimal(base_size = base_size, base_family = base_family) +
#     theme(
#       plot.title = element_text(face = "bold", size = base_size + 2, hjust = 0.5),
#       axis.title = element_text(size = base_size),
#       axis.text  = element_text(size = base_size * 0.9),
#       legend.title = element_text(size = base_size),
#       legend.text  = element_text(size = base_size * 0.9),
#       panel.grid.major = element_line(linewidth = 0.4),
#       panel.grid.minor = element_blank(),
#       plot.margin = margin(10, 10, 10, 10)
#     )
# }

# theme_set(theme_paper())
```

[4]:
```
#save.image("workspaces/afterCorrelations_wSTAR.RData")
load("workspaces/afterCorrelations_wSTAR_thr2percCells_10xHuman_8CLC.RData")
```

```
gc()
```

A matrix: 2 × 6 of type dbl

	used	(Mb)	gc trigger	(Mb)	max used	(Mb)
Ncells	33248296	1775.7	60333000	3222.2	33251622	1775.9
Vcells	299500060	2285.1	430405190	3283.8	299519875	2285.2

1 Gene Intersections

[5]:
```
library(rtracklayer)
# import the annotation used for snakemake
gene_annotation <- rtracklayer::import("/mnt/TEdataStorage_2T/snakemake_data/
  ↪references/hg38/gencode.v30.annotation.gtf.gz")
names(mcols(gene_annotation))
```

1. 'source' 2. 'type' 3. 'score' 4. 'phase' 5. 'gene_id' 6. 'gene_type' 7. 'gene_name' 8. 'level' 9. 'havana_gene' 10. 'transcript_id' 11. 'transcript_type' 12. 'transcript_name' 13. 'transcript_support_level' 14. 'tag' 15. 'havana_transcript' 16. 'exon_number' 17. 'exon_id' 18. 'ont' 19. 'protein_id' 20. 'ccdsid'

[6]:
```
gene_annotation_df <- as.data.frame(gene_annotation)

# Extract only exon entries
exons <- gene_annotation[gene_annotation$type == "exon"]
names(exons) <- exons$gene_name
```

[7]:
```
# keep only TEs that are in both SoloTE and Stellarscope
annotation_common <- na.omit(conversionTable)

cat("Classes not in SoloTE annotation: ",
    paste(setdiff(unique(conversionTable$class),␣
  ↪unique(annotation_common$class)), collapse = ", "),
    "\n")
```

```
Classes not in SoloTE annotation:  Satellite, Retroposon, NA, Unknown, RNA
```

[8]:
```
objTElist <- list()

objTElist[["SoloTE"]] <- objTE_soloTE
objTElist[["Stellarscope"]] <- objTE_stellarscope
objTElist[["STARsolo"]] <- objTE_STARsolo

objTElist
```

```
$SoloTE
An object of class Seurat
365333 features across 125 samples within 1 assay
Active assay: RNA (365333 features, 4000 variable features)
 3 layers present: counts, data, scale.data
```

```
 2 dimensional reductions calculated: pca, umap

$Stellarscope
An object of class Seurat
234752 features across 125 samples within 1 assay
Active assay: RNA (234752 features, 4000 variable features)
 3 layers present: counts, data, scale.data
 2 dimensional reductions calculated: pca, umap

$STARsolo
An object of class Seurat
224828 features across 125 samples within 1 assay
Active assay: RNA (224828 features, 4000 variable features)
 3 layers present: counts, data, scale.data
 2 dimensional reductions calculated: pca, umap
```

[9]:
```
# Compute overlaps to exons and save lists of loci
featuresList <- list()

bedList <- list()
grList <- list()
exonOverlapList <- list()
toolOverlappingGrList <- list()
toolOverlappingList <- list()

for(tool in names(objTElist)){
    if(tool == "SoloTE"){
        bedList[[tool]] <- conversionTable[conversionTable$soloteID %in%␣
 ↪Features(objTElist[[tool]]),
                            c("chr","start","end","strand","stellarscopeID")]
        print(nrow(bedList[[tool]]))
    }else{
        bedList[[tool]] <- conversionTable[conversionTable$stellarscopeID %in%␣
 ↪Features(objTElist[[tool]]),
                            c("chr","start","end","strand","stellarscopeID")]
        print(nrow(bedList[[tool]]))
    }

    bedList[[tool]] <- bedList[[tool]][!
 ↪duplicated(bedList[[tool]]$stellarscopeID),]
    bedList[[tool]] <- bedList[[tool]][!is.na(bedList[[tool]]$stellarscopeID),]
    featuresList[[tool]] <- bedList[[tool]]$stellarscopeID
    print(nrow(bedList[[tool]]))
    rownames(bedList[[tool]]) <- bedList[[tool]]$stellarscopeID
    # turn into GR object
    grList[[tool]] <- makeGRangesFromDataFrame(bedList[[tool]])
```

```
    exonOverlapList[[tool]] <- findOverlaps(grList[[tool]], exons, minoverlap =␣
 ↪10)

    toolOverlappingGrList[[tool]] <- grList[[tool]][unique(␣
 ↪exonOverlapList[[tool]]@from)]
    toolOverlappingList[[tool]] <- rownames(as.data.
 ↪frame(toolOverlappingGrList[[tool]]))
    #print(as.data.frame(toolOverlappingList[[tool]]))
}
```

```
[1] 364661
[1] 364272
[1] 234752
[1] 234752
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
"Each of the 2 combined objects has sequence levels not in the other:
  - in 'x': MU273330.1, MU273338.1, MU273339.1, MU273349.1, MU273375.1,
MU273387.1, GL000009.2, GL000194.1, GL000195.1, GL000218.1, GL000219.1,
GL000221.1, GL000224.1, GL000250.2, GL000251.2, GL000252.2, GL000253.2,
GL000254.2, GL000255.2, GL000256.2, GL000258.2, GL339449.2, GL383522.1,
GL383526.1, GL383530.1, GL383532.1, GL383549.1, GL383550.2, GL383551.1,
GL383557.1, GL383573.1, GL383574.1, GL383575.2, GL383576.1, GL383581.2,
GL383582.2, GL877875.1, GL877876.1, GL949746.1, GL949747.2, GL949752.1,
GL949753.2, KI270330.1, KI270435.1, KI270442.1, KI270706.1, KI270708.1,
KI270717.1, KI270719.1, KI270720.1, KI270722.1, KI270728.1, KI270729.1,
KI270733.1, KI270734.1, KI270741.1, KI270742.1, KI270743.1, KI270744.1,
KI270745.1, KI270763.1, KI270769.1, KI270772.1, KI270775.1, KI270777.1,
KI270779.1, KI270780.1, KI270784.1, KI270794.1, KI270795.1, KI270798.1,
KI270806.1, KI270808.1, KI270809.1, KI270810.1, KI270815.1, KI270816.1,
KI270831.1, KI270832.1, KI270844.1, KI270850.1, KI270853.1, KI270855.1,
KI270856.1, KI270857.1, KI270860.1, KI270862.1, KI270871.1, KI270878.1,
KI270879.1, KI270880.1, KI270897.1, KI270904.1, KI270905.1, KI270908.1,
KI270913.1, KI270924.1, KI270926.1, KI270927.1, KI270935.1, KI270938.1,
KQ031389.1, KQ090017.1, KQ090023.1, KQ458383.1, KQ458384.1, KQ983256.1,
KV575243.1, KV766198.1, KV880763.1, KZ208904.1, KZ208908.1, KZ208913.1,
KZ559103.1, KZ559105.1
  - in 'y': chrM
  Make sure to always combine/compare objects based on the same reference
  genome (use suppressWarnings() to suppress this warning)."
```

```
[1] 224828
[1] 224828
```

```
Warning message in .merge_two_Seqinfo_objects(x, y):
"Each of the 2 combined objects has sequence levels not in the other:
  - in 'x': ML143343.1, ML143368.1, MU273330.1, MU273331.1, MU273332.1,
MU273337.1, MU273338.1, MU273339.1, MU273340.1, MU273349.1, MU273356.1,
MU273357.1, MU273358.1, MU273375.1, MU273378.1, MU273387.1, MU273395.1,
```

MU273396.1, MU273397.1, GL000008.2, GL000009.2, GL000194.1, GL000195.1,
GL000205.2, GL000208.1, GL000213.1, GL000214.1, GL000218.1, GL000219.1,
GL000220.1, GL000221.1, GL000224.1, GL000250.2, GL000251.2, GL000252.2,
GL000253.2, GL000254.2, GL000255.2, GL000256.2, GL000257.2, GL000258.2,
GL339449.2, GL383518.1, GL383519.1, GL383520.2, GL383521.1, GL383522.1,
GL383526.1, GL383527.1, GL383528.1, GL383530.1, GL383532.1, GL383534.2,
GL383539.1, GL383541.1, GL383542.1, GL383545.1, GL383546.1, GL383549.1,
GL383550.2, GL383551.1, GL383552.1, GL383553.2, GL383554.1, GL383555.2,
GL383556.1, GL383557.1, GL383563.3, GL383565.1, GL383566.1, GL383567.1,
GL383568.1, GL383569.1, GL383570.1, GL383571.1, GL383572.1, GL383573.1,
GL383574.1, GL383575.2, GL383576.1, GL383577.2, GL383579.2, GL383580.2,
GL383581.2, GL383582.2, GL383583.2, GL582966.2, GL877875.1, GL877876.1,
GL949742.1, GL949746.1, GL949747.2, GL949748.2, GL949749.2, GL949750.2,
GL949751.2, GL949752.1, GL949753.2, JH159137.1, JH159146.1, JH636055.2,
KB021644.2, KB663609.1, KI270442.1, KI270538.1, KI270706.1, KI270707.1,
KI270708.1, KI270711.1, KI270712.1, KI270713.1, KI270717.1, KI270718.1,
KI270719.1, KI270720.1, KI270721.1, KI270722.1, KI270725.1, KI270726.1,
KI270727.1, KI270728.1, KI270729.1, KI270731.1, KI270733.1, KI270734.1,
KI270741.1, KI270742.1, KI270743.1, KI270744.1, KI270745.1, KI270748.1,
KI270749.1, KI270751.1, KI270758.1, KI270759.1, KI270760.1, KI270762.1,
KI270763.1, KI270765.1, KI270766.1, KI270767.1, KI270769.1, KI270770.1,
KI270772.1, KI270774.1, KI270775.1, KI270776.1, KI270777.1, KI270778.1,
KI270779.1, KI270780.1, KI270781.1, KI270782.1, KI270783.1, KI270784.1,
KI270786.1, KI270787.1, KI270788.1, KI270789.1, KI270790.1, KI270791.1,
KI270792.1, KI270793.1, KI270794.1, KI270795.1, KI270796.1, KI270797.1,
KI270798.1, KI270799.1, KI270801.1, KI270802.1, KI270803.1, KI270804.1,
KI270805.1, KI270806.1, KI270808.1, KI270809.1, KI270810.1, KI270811.1,
KI270812.1, KI270813.1, KI270814.1, KI270815.1, KI270816.1, KI270817.1,
KI270818.1, KI270819.1, KI270821.1, KI270822.1, KI270823.1, KI270824.1,
KI270825.1, KI270826.1, KI270829.1, KI270830.1, KI270831.1, KI270832.1,
KI270835.1, KI270838.1, KI270839.1, KI270840.1, KI270844.1, KI270846.1,
KI270847.1, KI270848.1, KI270849.1, KI270850.1, KI270851.1, KI270852.1,
KI270853.1, KI270854.1, KI270855.1, KI270856.1, KI270857.1, KI270858.1,
KI270859.1, KI270860.1, KI270861.1, KI270862.1, KI270863.1, KI270865.1,
KI270866.1, KI270867.1, KI270868.1, KI270869.1, KI270870.1, KI270871.1,
KI270872.1, KI270873.1, KI270874.1, KI270875.1, KI270876.1, KI270878.1,
KI270879.1, KI270880.1, KI270881.1, KI270882.1, KI270883.1, KI270885.1,
KI270887.1, KI270888.1, KI270889.1, KI270890.1, KI270891.1, KI270893.1,
KI270894.1, KI270895.1, KI270896.1, KI270897.1, KI270898.1, KI270899.1,
KI270900.1, KI270902.1, KI270903.1, KI270904.1, KI270905.1, KI270906.1,
KI270907.1, KI270908.1, KI270909.1, KI270910.1, KI270911.1, KI270912.1,
KI270913.1, KI270915.1, KI270916.1, KI270917.1, KI270919.1, KI270920.1,
KI270921.1, KI270922.1, KI270923.1, KI270924.1, KI270925.1, KI270926.1,
KI270927.1, KI270928.1, KI270929.1, KI270930.1, KI270931.1, KI270932.1,
KI270933.1, KI270934.1, KI270935.1, KI270936.1, KI270937.1, KI270938.1,
KN196477.1, KN196485.1, KN196486.1, KN538368.1, KQ031389.1, KQ031390.1,
KQ090013.1, KQ090014.1, KQ090015.1, KQ090017.1, KQ090018.1, KQ090019.1,
KQ090020.1, KQ090023.1, KQ090024.1, KQ090026.1, KQ090027.1, KQ458382.1,

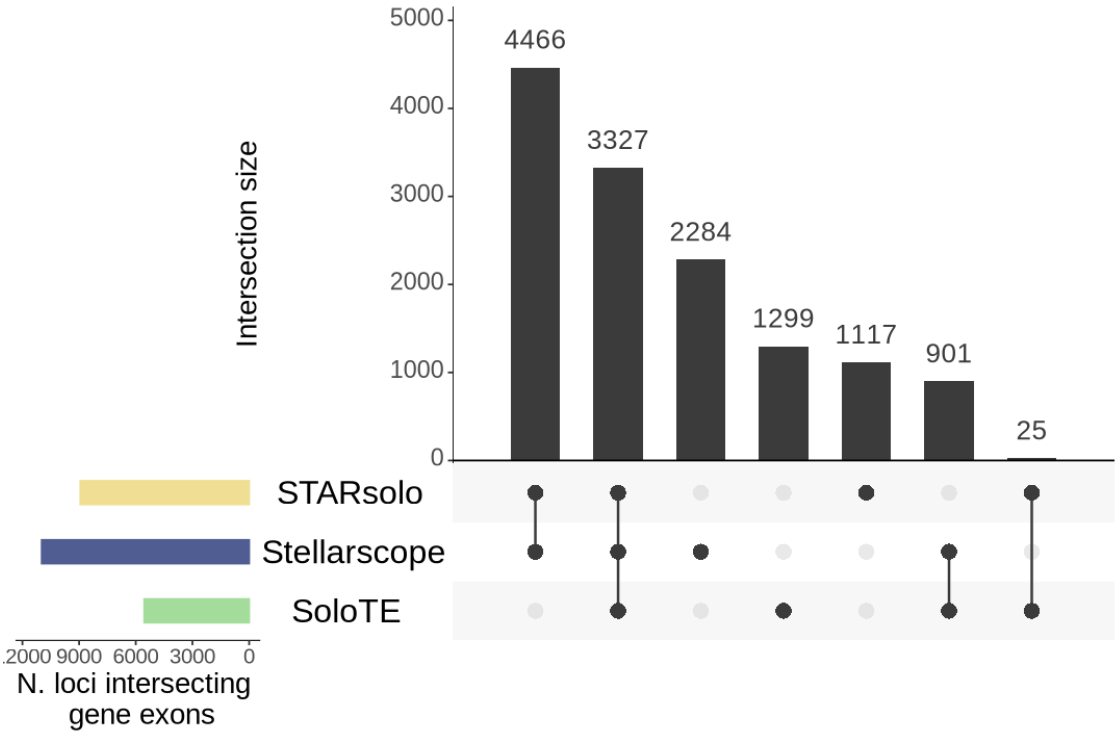
```
KQ458383.1, KQ458384.1, KQ458385.1, KQ458387.1, KQ458388.1, KQ759761.1,
KQ983255.1, KQ983256.1, KQ983258.1, KV575243.1, KV575247.1, KV575248.1,
KV575250.1, KV575252.1, KV575253.1, KV575254.1, KV575256.1, KV575257.1,
KV575258.1, KV575259.1, KV766193.1, KV766197.1, KV766198.1, KV880763.1,
KZ208904.1, KZ208907.1, KZ208908.1, KZ208909.1, KZ208913.1, KZ208918.1,
KZ208919.1, KZ208921.1, KZ559101.1, KZ559103.1, KZ559105.1, KZ559107.1,
KZ559110.1, KZ559111.1, KZ559112.1, KZ559114.1, KZ559116.1
  - in 'y': chrM
  Make sure to always combine/compare objects based on the same reference
  genome (use suppressWarnings() to suppress this warning)."
```

[10]:
```
colorTools
```

SoloTE '#A4DD9B' **Stellarscope** '#4f5d93' **STARsolo** '#f0df93'

[11]:
```
library(UpSetR)
options(repr.plot.width=9, repr.plot.height=6)

ph <- UpSetR::upset(fromList(toolOverlappingList),
    sets=names(toolOverlappingList), sets.bar.color=colorTools,
    keep.order = TRUE,
    text.scale = c(2, 2, 2, 1.75, 2.75, 2.5),
    order.by=c("freq"),
    point.size=4, mb.ratio = c(0.63, 0.37),
    set_size.show=F, set_size.scale_max= max(lengths(toolOverlappingList))+800,
    sets.x.label="N. loci intersecting \n gene exons",
    mainbar.y.label="Intersection size")

pdf(paste0("figures_", dataset_id, "/NlociIntersectingGeneExons_upset.pdf"),␣
 ↪width=9, height=6)
ph
dev.off()
ph
```

pdf: 2



[12]:
```
df <- NULL

for(tool in names(objTElist)){

    loci <- bedList[[tool]]$stellarscopeID
    lociOverlappingGenes <- toolOverlappingList[[tool]]

    df <- rbind(df, c(tool, length(loci), length(lociOverlappingGenes)))

}

df <-  as.data.frame(df)
df
```

A data.frame: 3 × 3

	V1 <chr>	V2 <chr>	V3 <chr>
	SoloTE	364272	5552
	Stellarscope	234752	10978
	STARsolo	224828	8935

[13]:
```
library(dplyr)
library(purrr)
library(tidyr)
```

```r
make_membership_table <- function(all_lists, gene_lists=NULL) {

  # All loci that appear in at least one tool
  all_loci <- unique(unlist(all_lists))

  # Presence/absence table (automatic)
  df <- tibble(locus = all_loci)

  for (nm in names(all_lists)) {
    df[[nm]] <- df$locus %in% all_lists[[nm]]
  }

  # Optional: add gene-overlap column
  if (!is.null(gene_lists)) {
    gene_loci <- unique(unlist(gene_lists))
    df$gene_overlap <- df$locus %in% gene_loci
  }

  df
}

df <- make_membership_table(featuresList, toolOverlappingList)
head(df)
```

```
Attaching package: ‘dplyr’


The following objects are masked from ‘package:GenomicRanges’:

    intersect, setdiff, union


The following object is masked from ‘package:GenomeInfoDb’:

    intersect


The following objects are masked from ‘package:IRanges’:

    collapse, desc, intersect, setdiff, slice, union


The following objects are masked from ‘package:S4Vectors’:

    first, intersect, rename, setdiff, setequal, union
```

```
The following objects are masked from ‘package:BiocGenerics’:

    combine, intersect, setdiff, union

The following objects are masked from ‘package:stats’:

    filter, lag

The following objects are masked from ‘package:base’:

    intersect, setdiff, setequal, union


Attaching package: ‘purrr’

The following object is masked from ‘package:GenomicRanges’:

    reduce

The following object is masked from ‘package:IRanges’:

    reduce


Attaching package: ‘tidyr’

The following object is masked from ‘package:S4Vectors’:

    expand
```

A tibble: 6 × 5

locus <chr>	SoloTE <lgl>	Stellarscope <lgl>	STARsolo <lgl>	gene_overlap <lgl>
AluSg-dup15	TRUE	TRUE	TRUE	FALSE
AluJb-dup71	TRUE	TRUE	FALSE	FALSE
L1MEg-dup7	TRUE	FALSE	FALSE	FALSE
L1PA4-dup20	TRUE	TRUE	TRUE	FALSE
L1MC1-dup4	TRUE	TRUE	FALSE	FALSE
AluSz-dup46	TRUE	FALSE	FALSE	FALSE

[14]:
```
df <- df %>%
  mutate(
    group = pmap_chr(select(., SoloTE:STARsolo), ~ {
      here <- c(...)
      tools <- names(here)[here]
      if (length(tools) == 0) return(NA_character_)
      paste(tools, collapse = "&")
    })
  )
```

[15]:
```
summary_df <- df %>%
  group_by(group) %>%
  summarise(
    N = n(),
    gene_N = sum(gene_overlap),
    percent_gene = round(100 * gene_N / N, 1),
    .groups = "drop"
  ) %>%
  arrange(desc(N))
```

[16]:
```
lengths(toolOverlappingList)
summary_df
```

SoloTE 5552 **Stellarscope** 10978 **STARsolo** 8935

A tibble: 7 × 4

group <chr>	N <int>	gene_N <int>	percent_gene <dbl>
SoloTE&Stellarscope&STARsolo	151286	3327	2.2
SoloTE	148702	1299	0.9
SoloTE&Stellarscope	61562	901	1.5
STARsolo	56776	1117	2.0
Stellarscope&STARsolo	14044	4466	31.8
Stellarscope	7860	2284	29.1
SoloTE&STARsolo	2722	25	0.9

[17]:
```
mix_colors <- function(cols) {
  cols_rgb <- t(col2rgb(cols))
  avg_rgb <- colMeans(cols_rgb)
  rgb(avg_rgb[1], avg_rgb[2], avg_rgb[3], maxColorValue = 255)
}

library(purrr)

# 1-way (original tool colors)
color_1way <- colorTools

# 2-way mixes
color_2way <- list(
```

```
  "SoloTE&Stellarscope" = mix_colors(colorTools[c("SoloTE","Stellarscope")]),
  "SoloTE&STARsolo"     = mix_colors(colorTools[c("SoloTE","STARsolo")]),
  "Stellarscope&STARsolo" = mix_colors(colorTools[c("Stellarscope","STARsolo")])
)

# 3-way mix
color_3way <- c(
  "SoloTE&Stellarscope&STARsolo" =
    mix_colors(colorTools[c("SoloTE","Stellarscope","STARsolo")])
)

# Combine everything
intersection_colors <- c(color_1way, color_2way, color_3way)
intersection_colors
```

$SoloTE '#A4DD9B'

$Stellarscope '#4f5d93'

$STARsolo '#f0df93'

$'SoloTE&Stellarscope' '#799D97'

$'SoloTE&STARsolo' '#CADE97'

$'Stellarscope&STARsolo' '#9F9E93'

$'SoloTE&Stellarscope&STARsolo' '#A1B395'

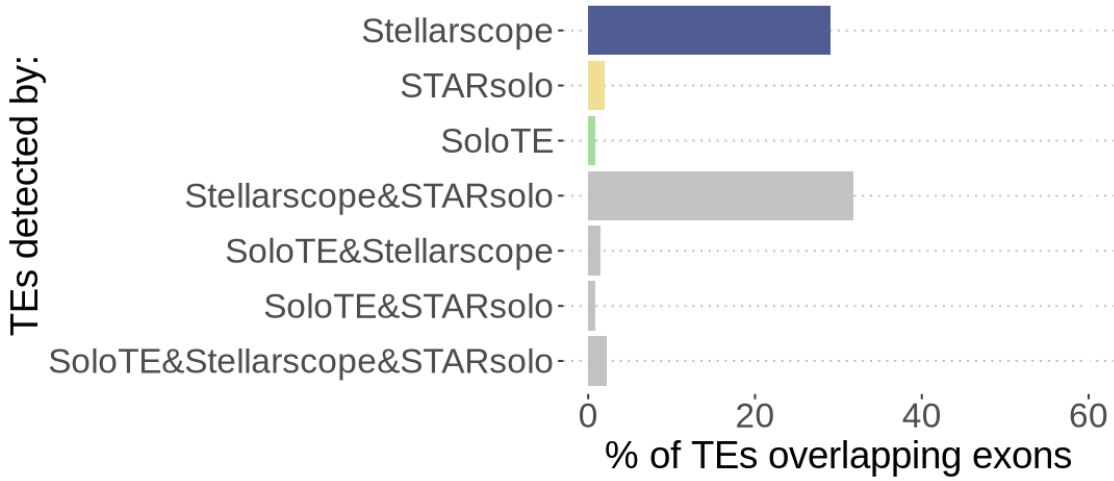
[18]:
```
summary_df
```

A tibble: 7 × 4

group <chr>	N <int>	gene_N <int>	percent_gene <dbl>
SoloTE&Stellarscope&STARsolo	151286	3327	2.2
SoloTE	148702	1299	0.9
SoloTE&Stellarscope	61562	901	1.5
STARsolo	56776	1117	2.0
Stellarscope&STARsolo	14044	4466	31.8
Stellarscope	7860	2284	29.1
SoloTE&STARsolo	2722	25	0.9

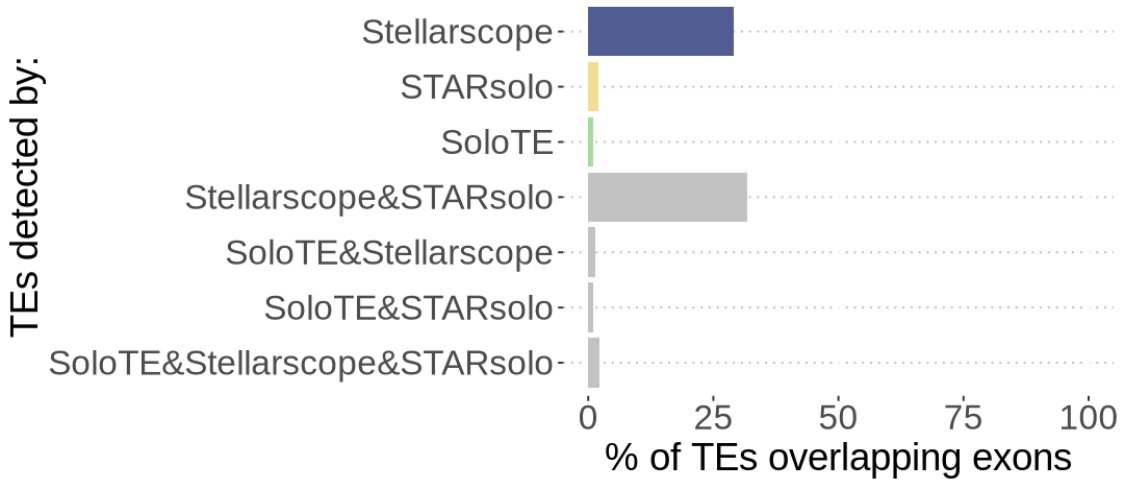
[19]:
```
options(repr.plot.width=9, repr.plot.height=4)
library(ggpubr)

df <- summary_df
# Step 1: compute size
df$size <- sapply(df$group, function(x) length(strsplit(x, "&")[[1]]))

# Step 2: get **unique groups** and sort by size
unique_groups <- unique(df$group)
```

```
group_levels <- unique_groups[order(sapply(unique_groups, function(x)␣
 ↪length(strsplit(x, "&")[[1]])))]

# descending within each size
df <- df %>%
  arrange(-size, percent_gene)  # or arrange(size, percent) for ascending

df$group <- factor(df$group, levels = df$group)  # preserve new order


ggplot(df , aes(y = group, x = percent_gene, fill = group)) +
  geom_col(alpha=1) +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("% of TEs overlapping exons") +
  ylab("TEs detected by:") +
  xlim(0,60) +
  theme_pubclean() +
  theme(text=element_text(size=22),
        axis.text=element_text(size=20),
        legend.position = "none"
        )
ggsave(paste0("figures_", dataset_id, "/percTEsOverlappingExons.pdf"), width=8,␣
 ↪height=6)
```



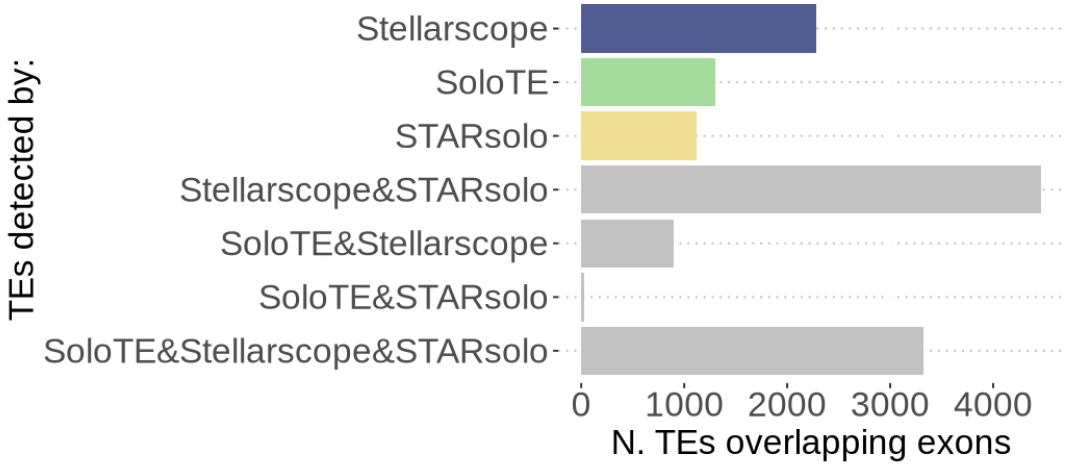
[20]:
```
options(repr.plot.width=9, repr.plot.height=4)

df <- summary_df
# Step 1: compute size
df$size <- sapply(df$group, function(x) length(strsplit(x, "&")[[1]]))
```

```
# Step 2: get **unique groups** and sort by size
unique_groups <- unique(df$group)
group_levels <- unique_groups[order(sapply(unique_groups, function(x)␣
 ↪length(strsplit(x, "&")[[1]])))]

# descending within each size
df <- df %>%
  arrange(-size, percent_gene)  # or arrange(size, percent) for ascending

df$group <- factor(df$group, levels = df$group)  # preserve new order


ggplot(df , aes(y = group, x = percent_gene, fill = group)) +
  geom_col() +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("% of TEs overlapping exons") +
  ylab("TEs detected by:") +
  xlim(0,100) +
  theme_pubclean() +
  theme(text=element_text(size=22),
        axis.text=element_text(size=20),
        legend.position = "none"
        )
ggsave(paste0("figures_", dataset_id, "/percTEsOverlappingExons_max100.pdf"),␣
 ↪width=8, height=4)
```



[21]:
```
# descending within each size
df <- df %>%
  arrange(-size, gene_N)  # or arrange(size, percent) for ascending
```

```
df$group <- factor(df$group, levels = df$group)  # preserve new order
ggplot(df , aes(y = group, x = gene_N, fill = group)) +
  geom_col() +
  scale_fill_manual(values = colorTools, na.value = "grey76") +
  xlab("N. TEs overlapping exons") +
  ylab("TEs detected by:") +
  theme_pubclean() +
  theme(text=element_text(size=20),
        axis.text=element_text(size=20),
        legend.position = "none",
        plot.margin = margin(t = 10, r = 30, b = 10, l = 10)  # top, right,␣
 ↪bottom, left
        )
ggsave(paste0("figures_", dataset_id, "/nTEsOverlappingExons.pdf"), width=8,␣
 ↪height=4)
```



[ ]: